Graph-Based Vulnerability Retrieval

Master's Thesis — Technical Progress

Lívia

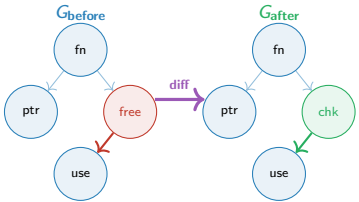
TU Munich · Siemens

April 14, 2026

Visual Recap

The problem: Given a new vulnerable function, find the most structurally similar historical CVEs — enabling automated patch suggestion.

Key insight: Code Property Graphs (CPGs) unify AST, CFG, and PDG into one representation — the *diff* between vulnerable and patched CPG isolates exactly what changed.



Changed node (red→green) isolated by semantic diff

System Architecture: End-to-End Pipeline

Data Layer
Dataset interface to CPG format.

Vector Layer
Structural diff $\rightarrow$ latent space projection.

Agent Layer
LLM reasoning augmented with retrieved CVE context.

From Source Code to Vulnerability Subgraph

Step 1 — CPG Export (Joern)

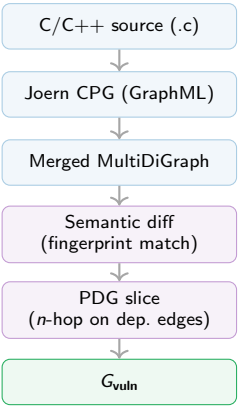
`joern-export --repr all` produces one `export.xml` per function covering AST, CFG, and PDG edges.

Step 2 — Graph Diff

Match nodes by Joern ID. *Identified problem:* Joern assigns non-deterministic IDs across runs — ID-based diff produced $G_{\text{vuln}} \approx G_{\text{before}}$, making all graphs indistinguishable.

Step 3 — CPG Slice

Extract the nodes that diverge and its neighbors with a hop-based of 1 scheme.



Unsupervised Graph Embedders

All embedders require no labelled training data. Output: L2-normalised 128-dim vector per function graph.

Structural embedders

NetLSD	Heat trace of the graph Laplacian. Permutation-invariant; captures global shape. *Collapses on graphs* < 5 *nodes.*
WL	Weisfeiler-Lehman colour refinement. Subtree kernel equivalent.
GIN	3-layer Graph Isomorphism Network with frozen random weights. Johnson-Lindenstrauss distance preservation via MLP non-linearity.
Combined	NetLSD + WL + GIN concatenated. More robust to individual collapse.

RAG Index Backends

Two interchangeable backends — identical `add` / `save` / `load` interface, swap via `config.yaml`.

FAISS Flat

- Exact cosine search (inner product on L2-normed vectors)
- Guaranteed correct top-k
- **Role:** correctness baseline in all experiments

HNSW

- Approximate nearest-neighbour via proximity graph
- $O(\log n)$ query time
- **Role:** test backend for $>$10k vectors

Experiment Design

Grid evaluated:
embedder $\times$ backend $\times$ graph variant

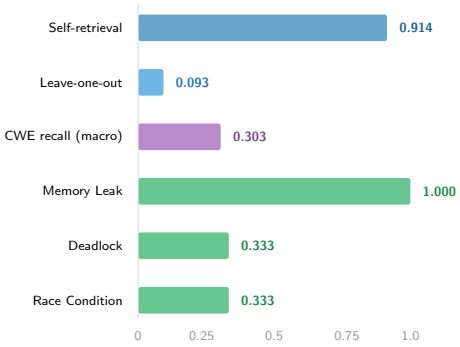
Metrics (all unsupervised — no labelled queries yet):

Hit@K Fraction of queries where correct CVE appears in top-K ($K \in \{1, 5, 10\}$).

MRR Mean reciprocal rank.

CWE recall Fraction of top-K sharing the query's CWE type — measures cluster quality.

Eff. dim Participation ratio of PCA eigenvalues — detects embedding collapse.



Best result: GIN on G_{vuln}, 75 pairs

Key Findings

ID-based diff was fatally flawed. Joern IDs are non-deterministic across runs. $G_{\text{vuln}} \approx G_{\text{before}}$ made all graphs structurally indistinct. *Fix: fingerprint matching by `(type, CODE)`.*

Embedding space collapsed. Effective dimension ≈ 1; pairwise cosine > 0.999. Causes: shared scaffold boilerplate nodes injected by wrapper, AST-edge dominance ($> 70\%$ of edges), and NetLSD/WL collapse on small graphs. *Fix: PDG-anchored slicing + scaffold node removal.*

CWE retrieval is structurally determined. CWEs whose patches *add or remove nodes* (Memory Leak, Deadlock) retrieve well. CWEs whose patches change *ordering or pointer state* (UAF, Race Condition) are near-undetectable by topology alone.

Semantic features are necessary. t-SNE of node-type histograms shows random CWE distribution — topology does not separate vulnerability classes.

Next Steps

Short term

4. Construct held-out query set with known CVE targets for honest precision/recall.
5. Change the calculation of graph differences. Instead of using the graph difference sets, look for a semantic difference (change of the nodes contents).

Current Goal

Find a sweetspot of the graph content and diff operation to then focus on G_{vuln} semantic embedder, then iterate.

Repository: `graph-rag/` Run: `uv run python main.py --mode experiment`